SIMATS ENGINEERING

ARTIFICIAL INTELLIGENCE – ASSESSMENT TOOL 2

Scenario-Based Assignment Answers

Course Code: CSA17 **Course Outcome: CO2**

Student Name: S. Aarathana

Register Number: 1921260021

Course Outcome (CO2):

Experiment with search and game-solving techniques such as Greedy Search, A*, CSP, Minimax and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently.

Code of Conduct

I certify that this submission is my original work and that I have followed the guidelines specified for this assessment.

Signature of the student: 

1. Emergency Medicine Delivery Using AI Drones

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

Greedy Best-First Search is an informed search algorithm that selects the path that appears closest to the goal according to a heuristic function. In this scenario, the drone uses straight-line distance to estimate how close it is to the medicine delivery destination.

Working:

1. The drone identifies possible routes to the destination.
2. It estimates the distance from each possible next location to the destination.
3. It selects the location with the smallest estimated distance.
4. It continues until it reaches the destination or discovers that a route is blocked.

Strengths:

- It can find a promising route quickly.
- It may require less search effort when a good route is easy to identify.
- It is useful when medicines must be delivered urgently.

Weaknesses:

- It considers estimated distance rather than total travel cost.
- A shorter route may contain flooded roads, dangerous weather or difficult terrain.
- It may choose a route that looks close but takes longer in reality.
- It may need to search again when a path becomes blocked.

Therefore, Greedy Best-First Search can be fast, but it may not find the safest or cheapest route.

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic.

A* is an informed search algorithm that considers both the actual cost of reaching a location and the estimated cost of reaching the destination.

Evaluation function: $f(n) = g(n) + h(n)$

- **g(n):** Actual cost from the starting point to the current location.
- **h(n):** Estimated cost from the current location to the destination.
- **f(n):** Estimated total cost of the route through that location.

Working:

1. The drone identifies possible routes.
2. It calculates actual travel costs, including distance, terrain and weather.
3. It estimates the remaining cost using the heuristic.
4. It selects the route with the lowest estimated total cost.
5. If a path becomes blocked, it updates route information and searches again.

A slightly longer route may be safer than a short route through a flooded area. A* can prefer the safer route if the cost function includes suitable penalties for risk and difficult conditions. Under the usual assumptions, A* can find an optimal route when its heuristic is admissible and the search is handled correctly.

iii. Analyze the impact of heuristic accuracy on both algorithms.

- **Greedy Best-First Search:** It depends heavily on the heuristic. A misleading straight-line estimate may lead the drone towards a blocked or dangerous route.
- **A* Search:** It uses both actual cost and heuristic estimates, so actual costs can help it avoid some misleading choices.
- **Accurate heuristic:** Both algorithms can make better-informed decisions.
- **Inaccurate heuristic:** Both may explore less suitable routes, but A* also considers the cost already incurred.

A heuristic that never overestimates the remaining cost is important when A* must guarantee an optimal solution under the usual assumptions.

iv. Discuss how dynamic changes (blocked paths) affect search performance.

- A previously available route may become inaccessible.
- Travel costs may change suddenly because of floods or weather.
- The drone may need to discard its current route and find an alternative.
- Repeated changes increase search time and computational effort.
- Old information may direct the drone towards unsafe locations.

The drone should receive real-time updates, check whether its selected path remains available and recalculate the route when necessary.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality and safety.

Recommendation: A* Search with real-time route updates. A* considers both actual travel cost and estimated remaining cost. The drone can include distance, weather, terrain and safety risks in its route-cost calculation.

- It can identify a low-cost route when the cost model is accurate.
- It considers both the distance travelled and the distance remaining.
- It can select safer routes when risks are included in the cost function.
- It can recalculate routes when new information becomes available.

A* may require more computation than Greedy Best-First Search. For strict real-time requirements, the drone may need efficient replanning and a time limit for route calculations. Overall, A* with real-time updates is a better balance of speed, cost and safety.

2. Smart-City Traffic Signal Optimization

Formulate this as an optimization problem and explain why traditional search is inefficient.

The smart city wants to minimize total waiting time, fuel consumption and traffic congestion. This can be formulated as an optimization problem.

Objective function: $Z = w1W + w2F + w3C$, to be minimized.

- W = total waiting time of vehicles.
- F = total fuel consumption.
- C = total traffic congestion.
- w1, w2 and w3 = weights representing the importance of each objective.
- Z = total cost to be minimized.

The system must choose green-light durations, signal sequences and timing plans. Traditional search may be inefficient because the number of possible combinations is very large, traffic changes continuously, evaluating every combination takes time and a solution that works at one time may fail at another.

i. Explain how Hill Climbing would work in this scenario and identify its limitations.

Hill Climbing is a local search algorithm that starts with an initial solution and repeatedly moves to a better neighbouring solution.

Working:

1. Start with an initial traffic signal timing plan.
2. Measure waiting time, fuel consumption and congestion.
3. Change one or more signal timings slightly.
4. Compare the new plan with the previous plan.
5. Accept the new plan if it improves the objective function.
6. Repeat until no better neighbouring solution is found.

For example, the system may increase the green-light duration at an intersection with long queues. If the total traffic cost decreases, it accepts the change.

Limitations:

- It may get stuck in a local optimum.
- It may fail to find the best solution for the whole city.
- It can struggle when many intersections affect one another.
- It may not adapt quickly enough to sudden traffic changes.
- Its result depends on the initial solution.

ii. Discuss issues like local maxima, plateaus and ridges with real-world interpretation.

- **Local maxima / local optima:** In this minimization problem, this is more precisely a local minimum. A timing plan may improve two intersections but still be worse than a different city-wide plan.
- **Plateaus:** Neighbouring solutions have almost the same evaluation value. Changing a signal by a few seconds may not noticeably change total waiting time, so the algorithm may stop.
- **Ridges:** A better solution may require coordinated changes. Improving one signal alone may increase congestion, while changing two connected signals together may improve traffic flow.

These issues can prevent Hill Climbing from finding a good overall solution.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

A. Simulated Annealing

Simulated Annealing is inspired by the cooling of metals. It sometimes accepts a worse solution so it can escape a local optimum.

1. Start with an initial traffic signal plan.
2. Generate a neighbouring plan.
3. Evaluate whether the new plan improves the objective.
4. Accept an improved plan.
5. Sometimes accept a worse plan with a probability controlled by temperature.

6. Gradually lower the temperature so accepting worse solutions becomes less likely.

This allows the system to explore different timing plans and avoid becoming trapped too early.

B. Genetic Algorithms

Genetic Algorithms are inspired by natural selection. They maintain a population of possible solutions.

1. Generate several possible traffic timing plans.
2. Evaluate each plan using waiting time, fuel consumption and congestion.
3. Select better-performing plans as parents.
4. Use crossover to combine parts of different plans.
5. Use mutation to introduce small random changes.
6. Repeat until a satisfactory solution is found.

Genetic Algorithms can explore many solutions and are useful when the search space is very large.

iv. Recommend the best approach and justify your choice based on scalability and adaptability.

Recommendation: Genetic Algorithm combined with real-time traffic data. A Genetic Algorithm is suitable because hundreds of intersections create a very large search space. It explores multiple candidate solutions and combines good timing plans.

- It searches several possible solutions at the same time.
- It is less dependent on a single starting solution.
- It can optimize multiple objectives.
- It can be run again when traffic patterns change.

Genetic Algorithms may require significant computation and do not guarantee the global optimum. Population size and the number of generations must be selected carefully. In practice, the system can generate timing plans and update them using live traffic data.

3. Autonomous Mars Rover Using Online Search

i. Explain why this problem requires an online search agent instead of offline search.

An online search agent makes decisions while interacting with an environment and discovering new information. An offline search agent generally plans using a model or map available before execution.

The Mars rover explores unknown terrain and does not have a complete map. Online search is needed because:

- The rover discovers obstacles as it moves.
- It has limited sensing capability.
- The terrain may differ from its initial expectations.
- It must update its knowledge during exploration.
- It must make decisions without knowing every route in advance.

For example, if the rover discovers a crater blocking its planned path, it can use the new information to choose another route. Therefore, online search is suitable for exploration in an unknown environment.

ii. Analyze the challenges of partial observability and dynamic environments.

Partial observability means that the rover cannot observe the entire environment at once.

- It may not know the location of every rock or crater.

- Its sensors may provide incomplete or uncertain information.
- It may choose a route without knowing all hazards.
- It must make decisions using limited information.

New information about the terrain may reveal that the previous route is unsuitable. Unexpected obstacles or changing conditions can require a new plan. The rover must balance exploration, safety and progress towards its goal.

To overcome these challenges, it should use sensor information, update its internal model and replan when new information changes the suitability of its route.

iii. Describe how the rover builds and updates its internal model.

The rover maintains an internal model representing the terrain it has explored and the information it has collected.

1. **Initial model:** Begin with the limited map or information available.
2. **Sensing:** Collect information about nearby rocks, craters and possible paths.
3. **Model updating:** Add newly discovered obstacles and traversable areas.
4. **Route selection:** Use the updated model to select the next movement.
5. **Movement:** Move towards the selected location while monitoring surroundings.
6. **Replanning:** If a hazard or blocked route is discovered, update the model and choose another path.

This continuous process allows the rover to learn about its environment and improve its navigation decisions.

iv. Compare online search with uninformed and informed search strategies.

Feature	Online search	Uninformed search	Informed search
Main idea	Makes decisions while exploring	Searches without a heuristic estimate	Uses a heuristic to guide search
Information	Learns during execution	Uses a represented problem space	Uses available information and estimates
Terrain knowledge	Can work with an incomplete map	Often assumes a sufficiently represented search space	Uses a heuristic to estimate progress
Adaptability	Can respond to new information	May need restarting or modification	May need replanning when conditions change
Example	Rover exploring unknown terrain	Breadth-First Search	A* Search

Online search describes how an agent searches while acting in an environment. Uninformed and informed search describe how a search is guided. These categories are not completely exclusive: informed techniques can also be adapted for online exploration.

v. Justify the most suitable approach considering uncertainty, adaptability and safety.

Recommendation: An online search agent with heuristic guidance and continuous replanning.

The rover should explore its surroundings, collect sensor information and update its internal model before choosing the next movement.

- It can make decisions without a complete map.
- It responds to newly discovered obstacles.
- Heuristic information can guide exploration towards sample locations.

- It can avoid known hazards and replan when a route becomes unsafe.

Safety should have high priority. The rover should reject paths that violate safety constraints, even if they appear shorter. An online search agent with heuristic guidance is suitable because it can learn about unknown terrain and adapt as new information becomes available.

4. University Exam Timetable Using Constraint Satisfaction Problems (CSP)

Formulate the problem as a Constraint Satisfaction Problem (CSP).

A Constraint Satisfaction Problem is an AI problem in which values are assigned to variables while satisfying a set of constraints. The university must schedule examinations without student clashes while considering hall availability, subject order, faculty availability and special accommodations.

A CSP consists of three main components: variables, domains and constraints.

i. Clearly define variables, domains and constraints with explanation.

A. Variables

Variables represent examinations that need to be scheduled. For example, E1 = Artificial Intelligence, E2 = Computer Networks and E3 = Database Management. Each examination must be assigned a suitable time slot and examination hall.

B. Domains

A domain is the set of possible values that can be assigned to a variable. For example, possible time slots may include morning, afternoon and evening if permitted by the university. The domain also includes available halls that satisfy capacity and accessibility requirements.

C. Constraints

- **No overlapping examinations:** A student must not have two examinations at the same time.
- **Limited examination halls:** The number of simultaneous examinations cannot exceed the available suitable halls.
- **Subject ordering:** If one subject must be conducted before another, their time slots must follow that order.
- **Faculty availability:** Examinations must follow the availability of required faculty members or invigilators.
- **Special accommodations:** Students requiring special facilities must be assigned suitable halls and arrangements.
- **Hall capacity:** Each hall must have enough seats for the students taking the examination.

The timetable is valid only when all required constraints are satisfied.

ii. Explain how backtracking search works in this scenario.

Backtracking assigns values to variables one at a time. If an assignment violates a constraint, the algorithm goes back and tries another value.

1. Select an examination that needs to be scheduled.
2. Assign an available time slot and hall.
3. Check whether the assignment satisfies all constraints.
4. If valid, move to the next examination.
5. If a constraint is violated, remove the assignment and try another slot or hall.

6. Continue until all examinations are scheduled successfully.

For example, if the same students take Artificial Intelligence and Computer Networks, both cannot be assigned to Monday morning. The algorithm rejects that assignment and tries another slot. Backtracking can find a valid timetable, although it may take considerable time for a large problem.

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency.

Constraint propagation reduces the number of possible assignments by removing values that cannot satisfy the constraints. Forward checking examines the remaining variables after each assignment.

1. Assign Artificial Intelligence to Monday morning.
2. Identify other examinations that cannot be held at that time because of student clashes.
3. Remove Monday morning from the domains of those conflicting examinations.
4. Continue scheduling with the reduced domains.
5. If any examination has no valid slot remaining, backtrack immediately.

Advantages:

- Reduces unnecessary searching.
- Detects conflicts early.
- Avoids repeated invalid assignments.
- Improves backtracking efficiency.

Forward checking makes timetable generation faster by identifying problems before the entire timetable is constructed.

iv. Analyze real-world challenges and justify the best strategy for scalability.

Timetable generation becomes difficult as the number of students, courses, halls and faculty members increases.

- A large number of possible schedules.
- Students appearing for different examinations.
- Limited halls and seating capacity.
- Faculty and invigilator availability.
- Special accommodation requirements.
- Last-minute changes to examination dates or resources.

Recommended strategy: Backtracking with constraint propagation and intelligent variable ordering. This approach checks constraints while assigning examinations and reduces the search space.

- Schedule the most constrained examinations first.
- Use forward checking to remove invalid options.
- Apply backtracking when conflicts occur.
- Update the timetable when availability changes.

For very large institutions, additional optimization methods may be needed. Overall, CSP using backtracking, forward checking and intelligent variable ordering is a suitable strategy for generating a valid timetable.

5. Strategic Game AI Using Minimax and Alpha-Beta Pruning

i. Explain how the Minimax algorithm models this problem.

Minimax is a game-search algorithm used in competitive decision-making. It assumes two players take turns: one tries to maximize the outcome and the other tries to minimize it.

In this scenario:

- **MAX player:** The AI selects the move with the highest possible evaluation.
- **MIN player:** The opponent is assumed to select moves that reduce the AI's advantage.

Working:

1. Examine the current game state.
2. Generate possible moves.
3. Build a game tree representing future moves.
4. Evaluate resulting game states.
5. At MAX levels, select the highest value.
6. At MIN levels, select the lowest value.
7. Choose the move that gives the best result against an optimal opponent.

For example, if Move A has a worst-case evaluation of 5 and Move B has a worst-case evaluation of 3, the AI chooses Move A. Minimax helps the AI consider possible responses from the opponent.

ii. Analyze the computational challenges of large game trees.

- **Large branching factor:** Many possible moves create many branches.
- **Increasing search depth:** Examining more future moves rapidly increases the number of states.
- **Time limitations:** The AI must choose a move within a limited time.
- **Memory requirements:** Storing and processing game states needs memory.
- **Complex evaluation:** Resources, unit strength, territory and threats must be considered.

Searching the entire game tree is often impractical. The AI needs methods that reduce computation while still selecting good moves.

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

Alpha-Beta pruning is an optimization technique used with Minimax. It avoids exploring branches that cannot affect the final decision.

- **Alpha (α):** The best value MAX can guarantee so far.
- **Beta (β):** The best value MIN can guarantee so far.

When $\alpha \geq \beta$, the remaining branches at that point can be pruned because they cannot improve the relevant decision.

Example:

1. The AI evaluates the first branch, MIN A.
2. MIN A chooses the smaller of 5 and 6, giving a value of 5.
3. MAX now has a guaranteed value of at least 5, so $\alpha = 5$.
4. The AI examines MIN B and finds a value of 4.

5. Since MIN B can already choose 4, MAX would not prefer this branch over the existing value of 5.
6. The remaining child of MIN B does not need to be evaluated.

Advantages:

- Reduces the number of game states evaluated.
- Saves computation time.
- Allows deeper searches within the same time limit.
- Produces the same Minimax decision at the same search depth and with the same evaluation values.

The effectiveness of pruning depends on the order in which moves are examined. Good move ordering can improve performance.

iv. Design an evaluation function and justify the factors considered.

An evaluation function estimates how favourable a game state is for the AI when it cannot search all the way to the end of the game.

Evaluation function: $E(s) = w1R + w2U + w3T + w4P - w5D$

- R = resource advantage.
- U = unit strength advantage.
- T = territory controlled.
- P = progress towards the game objective.
- D = damage or threat faced by the AI.
- w1, w2, w3, w4 and w5 = weights assigned to the factors.
- E(s) = evaluation score of the current state.

A higher score represents a more favourable position for the AI.

- **Resource advantage:** More resources help produce units and improve capabilities.
- **Unit strength:** A stronger army can improve the chances of winning battles.
- **Territory controlled:** Important locations provide strategic advantages.
- **Objective progress:** Moves closer to victory should receive higher scores.
- **Damage and threats:** Heavy losses or dangerous enemy positions should reduce the score.

The weights should be adjusted according to the rules and objectives of the particular game. This is a proposed design, not a universal formula for every strategy game.

v. Recommend a strategy for balancing optimality and real-time performance.

Recommendation: Minimax with Alpha-Beta pruning, depth-limited search and a heuristic evaluation function.

1. Generate possible moves from the current game state.
2. Search the game tree to a selected depth using Minimax.
3. Apply Alpha-Beta pruning to avoid unnecessary branches.
4. Evaluate remaining states using the heuristic evaluation function.
5. Prioritize promising moves to improve pruning efficiency.
6. Stop searching when the time limit is reached and select the best move found so far.

A deeper search can improve decision quality but requires more computation. Search depth should be selected according to the available time and processing power. This combination offers a practical balance between decision quality and real-time performance.

Overall Conclusion

Artificial Intelligence uses different search and optimization techniques to solve complex real-world problems. Greedy Best-First Search and A* help find routes, Hill Climbing and Genetic Algorithms optimize traffic signals, online search supports exploration in unknown environments, CSP techniques generate valid examination timetables, and Minimax with Alpha-Beta pruning supports strategic game decisions.

Each algorithm has its own advantages and limitations. The most suitable method depends on the problem, available information, size of the search space, safety requirements and time constraints. By selecting appropriate algorithms and combining them with reliable data, heuristics and constraint handling, AI systems can make more efficient and effective decisions.